

Phase 1: Network & Splunk Setup

Objective:

The goal for this phase was to build the basic SOC/Cyber Analysis lab environment and outline the purpose of each virtual machine before configuring log forwarding.

Environment:

The lab was created in VMware using two virtual machines:

- **Ubuntu VM** – Runs Splunk Enterprise and acts as the central SIEM server.
- **Windows 10 VM** – Acts as the monitored endpoint and runs the Splunk Universal Forwarder.
- **VMware VMnet8 (NAT)** – Provides network connectivity between the virtual machines.
- **TCP Port 9997** – Used by the Windows Universal Forwarder to send log data to splunk Enterprise.

Splunk Enterprise Configuration:

Splunk Enterprise was installed on the Ubuntu VM and configured as the central location for receiving, indexing, and analyzing security logs. The receiving service was configured to listen on TCP port 9997.

Windows Endpoint Configuration:

The Splunk Universal Forwarder was installed on Win 10 VM. Its purpose is to collect selected Windows event logs and forward them to the Ubuntu Splunk Enterprise server for centralized monitoring and analysis.

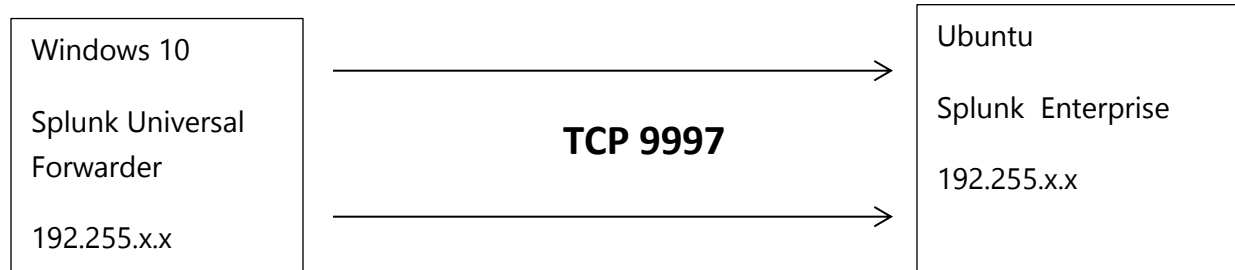
Windows Endpoint Configuration:

Once Configured, Windows security events would travel from the Win 10 endpoint through the Universal Forwarder to Splunk Enterprise, where the events could be searched and investigated using SPL (Search Processing Language).

Phase 2: Windows Log Forwarding

Network:

Before moving forward, I needed to establish communication between the Win 10 endpoint and Ubuntu Splunk Enterprise server using the Splunk Universal Forwarder.



Problem: Windows 10 Universal Forwarder could not establish a TCP connection to the Ubuntu Splunk Enterprise server. ICMP ping communication was successful, but TCP port 9997 was unreachable.

```
PS C:\Windows\system32> Test-NetConnection 192.255.x.x -Port 9997
WARNING: TCP connect to (192.255.x.x : 9997) failed
```

```
ComputerName      : 192.255.x.x
RemoteAddress     : 192.255.x.x
RemotePort        : 9997
InterfaceAlias    : Ethernet0
SourceAddress     : 192.255.x.x
PingSucceeded     : True
PingReplyDetails (RTT) : 0 ms
TcpTestSucceeded  : False
```

Root Cause: Firewall rules on the Ubuntu server were preventing inbound TCP connections. Although UFW initially appeared inactive, firewall rules were still present and affected TCP connectivity.

Trouble Shooting: PowerShell Test-Net Connection was used from the Win 10 endpoint to verify TCP connectivity to port 9997. On Ubuntu, “ss” was used to confirm that splunk was actively listening on port 9997. Netcat (nc) Was also used to verify that the splunk receiving port was accessible locally and through the Ubuntu network interface.

- **PowerShell:**
Test-NetConnection 192.255.x.x -Port 9997
- **Bash:**
Sudo ss -tlnp | grep 9997
- **Bash:**
Nc -vz 192.255.109.130 9997

Resolution: I configured the Ubuntu firewall to allow inbound TCP traffic on port 9997. After applying the firewall rule, I repeated the connectivity test from Win 10 and confirmed “TcpTestSucceeded : True”. The Splunk Universal Forwarder was then able to establish an active connection to the Ubuntu Splunk Enterprise server and successfully forward Windows Security Event Logs.

Forwarder Configuration: The Windows 10 Splunk Universal Forwarder was configured to send data to the Ubuntu Splunk Enterprise server TCP port 9997. The Forward server was added using the Ubuntu server's IP address, and the connection was then verified as active.

Bash:

```
sudo ufw allow 9997/tcp
```

- **Command Prompt:**

```
splunk.exe add forward-server 192.255.x.x:9997
```

Result:

```
PS C:\Windows\system32> Test-NetConnection 192.255.x.x -Port 9997
```

ComputerName : 192.255.x.x

RemoteAddress : 192.255.x.x

RemotePort : 9997

InterfaceAlias : Ethernet0

SourceAddress : 192.255.x.x

PingSucceeded : True

PingReplyDetails (RTT) : 0 ms

TcpTestSucceeded : True

i	Time	Event
		ComputerName=DESKTOP-2PHC8LJ Show all 19 lines host = DESKTOP-2PHC8LJ source = WinEventLog:System sourcetype = WinEventLog:System
>	8/9/26 7:15:44.185 PM	08/09/2026 07:15:44.185 PM LogName=System EventCode=10016 EventType=3 ComputerName=DESKTOP-2PHC8LJ Show all 19 lines host = DESKTOP-2PHC8LJ source = WinEventLog:System sourcetype = WinEventLog:System
>	8/9/26 7:15:32.465 PM	08/09/2026 07:15:32.465 PM LogName=System EventCode=10016 EventType=3 ComputerName=DESKTOP-2PHC8LJ Show all 19 lines host = DESKTOP-2PHC8LJ source = WinEventLog:System sourcetype = WinEventLog:System
>	8/9/26 7:15:14.390 PM	08/09/2026 07:15:14.390 PM LogName=System EventCode=24579 EventType=4 ComputerName=DESKTOP-2PHC8LJ Show all 12 lines host = DESKTOP-2PHC8LJ source = WinEventLog:System sourcetype = WinEventLog:System
>	8/9/26 7:15:14.386 PM	08/09/2026 07:15:14.386 PM LogName=System EventCode=24577 EventType=4 ComputerName=DESKTOP-2PHC8LJ Show all 12 lines host = DESKTOP-2PHC8LJ source = WinEventLog:System sourcetype = WinEventLog:System
>	8/9/26 7:15:14.332 PM	08/09/2026 07:15:14.332 PM LogName=System EventCode=24576 EventType=4 ComputerName=DESKTOP-2PHC8LJ

Phase 3: Failed Login Attempts

Investigation Result:

Splunk detected Windows event ID 4625 from the Windows 10 endpoint. The failed authentication targeted the local account and used Logon Type 2, including an interactive sign-in attempt at the workstation. The failure reason and sub-status of the code confirmed that the login failed because an incorrect password was entered.

Figure 1: Windows event ID 4625 captured in Splunk, Showing a failed interactive logon attempt for the user account due to an incorrect password

```
8/9/26      08/09/2026 09:43:00.425 PM
9:43:00.425 PM  LogName=Security
                EventCode=4625
                EventType=0
                ComputerName=DESKTOP-2PHC8LJ
                SourceName=Microsoft Windows security auditing.
                Type=Information
                RecordNumber=4184
                Keywords=Audit Failure
                TaskCategory=Logon
                OpCode=Info
                Message=An account failed to log on.

Subject:
    Security ID:      S-1-5-18
    Account Name:     DESKTOP-2PHC8LJ$
    Account Domain:   WORKGROUP
    Logon ID:         0x3E7

Logon Type:      2

Account For Which Logon Failed:
    Security ID:      S-1-0-0
    Account Name:     NIKO
    Account Domain:   DESKTOP-2PHC8LJ

Failure Information:
    Failure Reason:   Unknown user name or bad password.
    Status:           0xC000006D
    Sub Status:       0xC000006A

Process Information:
    Caller Process ID: 0x524
    Caller Process Name: C:\Windows\System32\svchost.exe

Network Information:
    Workstation Name:  DESKTOP-2PHC8LJ
    Source Network Address: 127.0.0.1
    Source Port:       0

Detailed Authentication Information:
```

Conclusion: The event confirmed a failed interactive login attempt against the local user account. The Windows Security log Showed Event ID 4625, Logon Type 2, and a sub-status indicating an incorrect password. This demonstrated that Splunk was successfully collection and displaying authentication failures from the Windows 10 endpoint.

Phase 4: Repeated Failed Login Detection

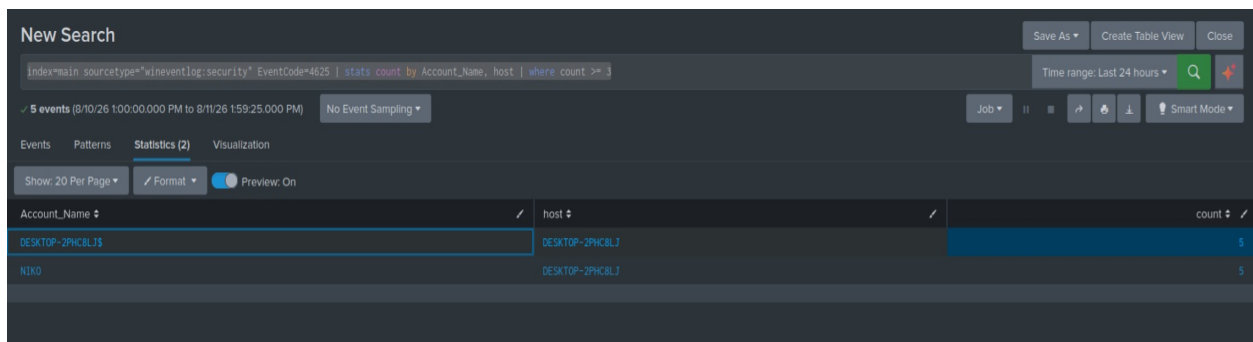
Detection Rule: My goal is to identify possible brute-force or password-guessing activity by looking for multiple ID 4625 events.

index=main sourcetype="WinEventLog:Security" EventCode=4625 | stats count by Account_Name, host | where count >= 3

EventCode=4625: Finds failed Windows logins.

Status count By Account_Name, host: Counts failures by user and computer.

Where count >= 3: Only shows accounts with at least three failed attempts.



New Search

index=main sourcetype="WinEventLog:Security" EventCode=4625 | stats count by Account_Name, host | where count >= 3

Time range: Last 24 hours

5 events (8/10/26 10:00:00 PM to 8/11/26 1:59:25 000 PM) No Event Sampling

Events Patterns Statistics (2) Visualization

Show: 20 Per Page Format Preview: On

Account_Name	host	count
DESKTOP-2PHC8LJ	DESKTOP-2PHC8LJ	5
NLKO	DESKTOP-2PHC8LJ	5

i	Time	Event
>	8/10/26 10:01:56.983 PM	08/10/2026 10:01:56.983 PM LogName=Security EventCode=4625 EventType=0 ComputerName=DESKTOP-2PHC8LJ Show all 61 lines host = DESKTOP-2PHC8LJ source = WinEventLog:Security sourcetype = WinEventLog:Security
>	8/10/26 10:01:54.996 PM	08/10/2026 10:01:54.996 PM LogName=Security EventCode=4625 EventType=0 ComputerName=DESKTOP-2PHC8LJ Show all 61 lines host = DESKTOP-2PHC8LJ source = WinEventLog:Security sourcetype = WinEventLog:Security
>	8/10/26 10:01:51.789 PM	08/10/2026 10:01:51.789 PM LogName=Security EventCode=4625 EventType=0 ComputerName=DESKTOP-2PHC8LJ Show all 61 lines host = DESKTOP-2PHC8LJ source = WinEventLog:Security sourcetype = WinEventLog:Security
>	8/10/26 10:01:50.034 PM	08/10/2026 10:01:50.034 PM LogName=Security EventCode=4625 EventType=0 ComputerName=DESKTOP-2PHC8LJ Show all 61 lines host = DESKTOP-2PHC8LJ source = WinEventLog:Security sourcetype = WinEventLog:Security

Conclusion: Splunk identified multiple failed authentication attempts against the same account, demonstrating how repeated login failures can be grouped and detected as potentially suspicious activity.

Phase 5: Alert Creation

Objective:

Convert the repeated failed-login detection into an automated alert so potentially suspicious authentication activity can be identified without manually running the SPL search.

Alert Creation: The failed-login detection search was converted into a scheduled Splunk alert. The alert checks for accounts with three or more failed authentication attempts within the selected time window and flags the activity for investigation.

For creating an alert screen I filled out like this:

Title: Repeated Failed Login Attempts

Permissions: Private

Alert Type: Scheduled

Schedule: Cron -> (* / 5 * * * *) – Runs every 5 min

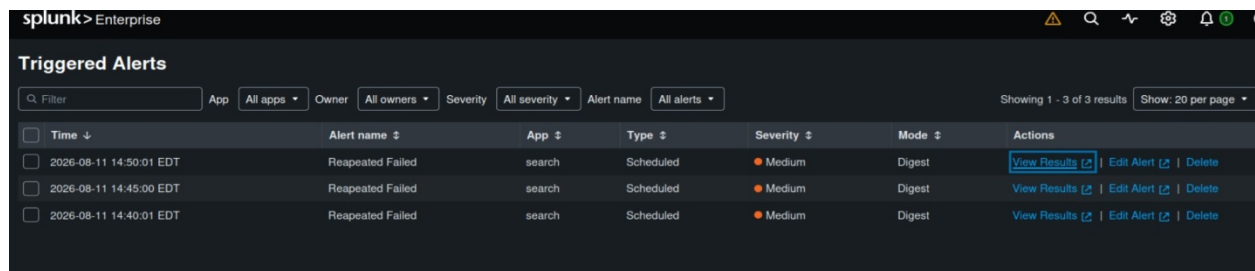
Trigger alert when: Number of results

Greater than: 0

Trigger: Once

Trigger Action: Add to trigger alerts -> **Severity:** Medium

The alert runs every 5 minutes and checks for accounts with three or more failed login attempts during the selected time window. If the search returns a result, Splunk triggers the alert for further investigation.



The screenshot shows the Splunk Enterprise web interface. At the top, there's a navigation bar with the Splunk logo and 'Enterprise' text. Below it, the 'Triggered Alerts' section is active. It features a search filter and several dropdown menus for filtering by App, Owner, Severity, Alert name, and All alerts. A table displays three triggered alerts, all with the name 'Repeated Failed', app 'search', type 'Scheduled', and severity 'Medium'. Each alert entry includes a checkbox, a timestamp, and a 'View Results' link. The table also shows the mode as 'Digest'.

☐	Time ↓	Alert name ↓	App ↓	Type ↓	Severity ↓	Mode ↓	Actions
☐	2026-08-11 14:50:01 EDT	Repeated Failed	search	Scheduled	Medium	Digest	View Results Edit Alert Delete
☐	2026-08-11 14:45:00 EDT	Repeated Failed	search	Scheduled	Medium	Digest	View Results Edit Alert Delete
☐	2026-08-11 14:40:01 EDT	Repeated Failed	search	Scheduled	Medium	Digest	View Results Edit Alert Delete

Result: The alert was configured to run every 5 minutes and trigger when the repeated failed-login detection returned one or more results. After generating multiple failed authentication attempts on the Windows 10 endpoint, Splunk successfully triggered the alert and recorded the activity for investigation.

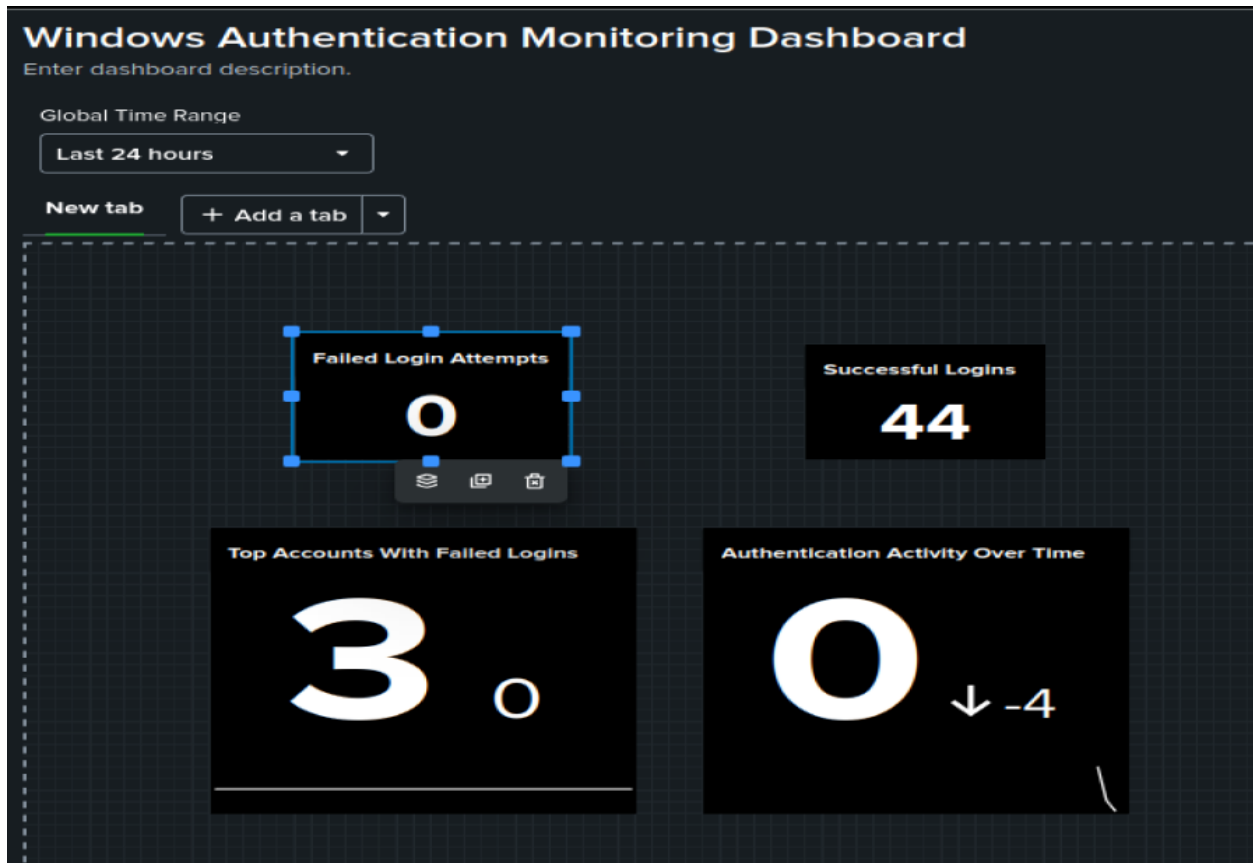
Phase 6: Dashboard Creation

Objective:

Create a Splunk dashboard to provide a centralized view of windows authentication activity and make repeated login behavior easier to monitor.

Dashboard Panels: The Dashboard includes failed login attempts, successful logins, top accounts with failed logins, and authentication activity over time. These visualizations allow an analyst to quickly identify unusual authentication patterns without manually running separate searches.

Result: The Dashboard successfully displayed Windows authentication data from the monitored endpoint and provided a consolidated view for ongoing monitoring and investigation.



Phase 7: Sysmon/Advanced Endpoint Monitoring

Objective:

Expand Windows endpoint monitoring by integrating Sysmon with the Splunk Universal Forwarder. Sysmon provides detailed endpoint telemetry that can assist with detecting and investigating suspicious system activity.

Issue Encountered: After configuring the Splunk Universal Forwarder to collect the Microsoft-Windows-Sysmon/Operational event channel, Sysmon events initially didn't appear in Splunk. Other Windows logs, including Security, System, and Application, Were being forward successfully.

Investigation: I first confirmed that Sysmon itself was functioning by verifying that the Microsoft-Windows-Sysmon/Operational event log existed and contained thousands of events. This ruled out a Sysmon installation or logging failure. I then verified that the Splunk Universal Forwarder was actively connected to the Splunk server and that other Windows logs, including Security, System, and Application, were being indexed successfully. This ruled out a general network or forwarding issue.

Next, I reviewed the Universal Forwarder configuration using btool and confirmed that the Sysmon input was enabled with disabled = 0 and configured to send data to the main index. Since the configuration was present but Sysmon events were still absent from Splunk, I reviewed the forwarder logs and service configuration. The investigation showed that the issue was isolated to the Universal Forwarder's ability to access and subscribe to the Sysmon Windows Event Log channel. After verifying that the SplunkForwarder service was running under NT SERVICE\SplunkForwarder , I granted the service account access to the Windows Event Log Readers group and restarted the forwarder. Sysmon events then began appearing in Splunk.

Resolution: I granted the Splunk Forwarder Service account access to the Windows Event Log Readers group and restarted the Splunk Universal Forwarder service. After restarting the service, Sysmon events began successfully forwarding to the Splunk server.

Verification: I verified the configuration:

```
.\splunk\splunk.exe btool inputs list -debug | Select-String "Sysmon" -Context 3,10
```

The Configuration showed:

```
[WinEventLog://Microsoft-Windows-Sysmon/Operational]
```

```
disabled = 0
```

```
index = main
```

I then verified ingestion in Splunk. The search returned Sysmon events from the Windows endpoint, including Event ID 1(Process Creation):

```
LogName=Microsoft-Windows-Sysmon/Operational
```

```
EventCode=1
```

```
ComputerName=DESKTOP-2PHC8LJ
```

```
source=WinEventLog:Microsoft-Windows-Sysmon/Operational
```

The search returned Sysmon events from the Windows endpoint, including Event ID 1 (Process Creation), confirming that the complete logging pipeline was operational.

Result: Win Endpoint -> Sysmon -> Universal Forwarder -> Splunk Server -> SIEM Search & Analysis

Figure 7.1-Successful Sysmon Event Ingestion

i	Time	Event
		ComputerName=DESKTOP-2PHC8LJ Show all 21 lines host = DESKTOP-2PHC8LJ : source = WinEventLog:Microsoft-Windows-Sysmon/Operational sourcetype = WinEventLog:Microsoft-Windows-Sysmon/Operational
>	8/17/26 12:09:11.056 PM	08/17/2026 12:09:11.056 PM LogName=Microsoft-Windows-Sysmon/Operational EventCode=1 EventType=4 ComputerName=DESKTOP-2PHC8LJ Show all 38 lines host = DESKTOP-2PHC8LJ : source = WinEventLog:Microsoft-Windows-Sysmon/Operational sourcetype = WinEventLog:Microsoft-Windows-Sysmon/Operational
>	8/17/26 12:09:10.336 PM	08/17/2026 12:09:10.336 PM LogName=Microsoft-Windows-Sysmon/Operational EventCode=5 EventType=4 ComputerName=DESKTOP-2PHC8LJ Show all 21 lines host = DESKTOP-2PHC8LJ : source = WinEventLog:Microsoft-Windows-Sysmon/Operational sourcetype = WinEventLog:Microsoft-Windows-Sysmon/Operational

Phase 8: Security Event Simulation & Incident Analysis

Objective:

The Objective of this phase was to conduct controlled security testing within the isolated lab environment and analyze the resulting endpoint activity using Sysmon and Splunk.

This Phase focused on Identifying suspicious process execution, reviewing command-line activity, correlating security events, and determining whether the observed activity represented expected or potentially suspicious behavior.

Controlled Test Activity: To generate a known event for analysis, I executed a controlled PowerShell command on the Windows endpoint.

Powershell.exe –NoProfile -ExecutionPolicy Bypass –Command "Write-Output 'SOC-LAB-TEST-001'"

The Unique identifier SOC-LAB-TEST-001 Was included so the resulting process could be located and correlated in Splunk

The command was executed only within the isolated lab environment.

Figure 8.1-Controlled PowerShell Activity

```
PS C:\Windows\system32> powershell.exe -NoProfile -ExecutionPolicy Bypass -Command "Write-Output 'SOC-LAB-TEST-001'"
SOC-LAB-TEST-001
PS C:\Windows\system32>
```

Detection: Sysmon recorded the PowerShell execution as Event ID 1(Process Creation). The Splunk Universal Forwarder collected the Sysmon event and forwarded it to the Splunk server for analysis.

I searched the Sysmon telemetry using:

Index = main sourcetype="WinEventlog:Microsoft-Windows-Sysmon/Operational" EventCode=1 "SOC-LAB-TEST-001"

The search returned the process associated with the controlled activity, confirming that the endpoint monitoring pipeline successfully captured and forwarded the event.

Figure 8.2-PowerShell Event Detected in Splunk

Format ▾ Show: 20 Per Page ▾ View: List ▾		
i	Time	Event
>	8/18/26 11:47:45.826 AM	08/18/2026 11:47:45.826 AM ... 22 lines omitted ... Company: Microsoft Corporation OriginalFileName: PowerShell.EXE CommandLine: "C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -NoProfile -ExecutionPolicy Bypass -Command "Write-Output 'SOC-LAB-TEST-001'" CurrentDirectory: C:\Windows\system32\ Show all 38 lines host = DESKTOP-2PHCBLJ source = WinEventLog:Microsoft-Windows-Sysmon/Operational sourcetype = WinEventLog:Microsoft-Windows-Sysmon/Operational

Investigation and Analysis: The detected PowerShell process was reviewed as potentially suspicious because PowerShell is a legitimate Windows administration utility that can also be abused during malicious activity.

The Command line contained:
Powershell.exe –NoProfile –ExecutionPolicy Bypass

The use of -ExecutionPolicy Bypass warranted further investigation before the activity could be classified as benign.

Figure 8.3 – Sysmon Process Creation Event Analysis

i	Time	Event
>	8/18/26 11:47:45.826 AM	<pre>08/18/2026 11:47:45.826 AM LogName=Microsoft-Windows-Sysmon/Operational EventCode=1 EventType=4 ComputerName=DESKTOP-2PHC8LJ User=NOT_TRANSLATED Sid=S-1-5-18 SidType=0 SourceName=Microsoft-Windows-Sysmon Type=Information RecordNumber=17713 Keywords=None TaskCategory=Process Create (rule: ProcessCreate) OpCode=Info Message=Process Create: RuleName: - UtcTime: 2026-08-18 15:47:45.816 ProcessGuid: {26f069e3-7eal-6a84-3004-000000001800} ProcessId: 2916 Image: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe FileVersion: 10.0.19041.3996 (WinBuild.160101.0800) Description: Windows PowerShell Product: Microsoft Windows® Operating System Company: Microsoft Corporation OriginalFileName: PowerShell.EXE CommandLine: "C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -NoProfile -ExecutionPolicy Bypass -Command "Write-Output 'SOC-LAB-TEST-001'" CurrentDirectory: C:\Windows\system32\ User: DESKTOP-2PHC8LJ\NIKO LogonGuid: {26f069e3-6c69-6a84-0ec3-010000000000} LogonId: 0x1C30E TerminalSessionId: 1 IntegrityLevel: High Hashes: SHA256=978500180DCF755EDDB8AF294A373C0B87B2498660F724E76C4D53F9C217C7A3 ParentProcessGuid: {26f069e3-6de9-6a84-9101-000000001800} ParentProcessId: 4540 ParentImage: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe ParentCommandLine: "C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" ParentUser: DESKTOP-2PHC8LJ\NIKO Collapse host = DESKTOP-2PHC8LJ source = WinEventLog:Microsoft-Windows-Sysmon/Operational sourcetype = WinEventLog:Microsoft-Windows-Sysmon/Operational</pre>

To make the relevant investigation fields easier to analyze, I created a Splunk table containing:

```
index=main sourcetype="WinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=1
"SOC-LAB-TEST-001" | table _time ComputerName User Image CommandLine ProcessId
ParentImage ParentCommandLine ParentProcessId Hashes
```

This allowed the process execution information to be reviewed in a structured format.

Figure 8.4 – Process Investigation Table

Show: 20 Per Page ▾ Format ▾ Preview: On						
ComputerName	User	Image	CommandLine	ProcessId	ParentImage	ParentComm
DESKTOP-2PHC8LJ	NOT_TRANSLATED	C:	"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe"	2916	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	C:
DESKTOP-2PHC8LJ\NIKO	DESKTOP-2PHC8LJ\NIKO	\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	\Windows\System32\WindowsPowerShell\v1.0\powershell.exe -NoProfile -ExecutionPolicy Bypass -Command "Write-Output 'SOC-LAB-TEST-001'"		\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	\Windows/Sy

Determination: After reviewing the event and correlating the command line with the controlled activity performed during the lab, the event was determined to be authorized test activity.

The SOC-LAB-TEST-001 identifier provided direct correlation between the activity intentionally generated on the Windows endpoint and the event detected in Splunk.

Classification: Authorized / Benign Test Activity

Severity: Low / Informational

Containment Required: No

Although the PowerShell execution contained characteristics that warranted investigation, the available evidence showed that the activity originated from the controlled lab test rather than unauthorized or malicious execution.

Result: Phase 8 successfully demonstrated the complete SOC monitoring and investigation workflow:

Windows Endpoint → PowerShell Activity → Sysmon → Splunk Universal Forwarder → Splunk SIEM → Event Detection → Investigation → Determination

The investigation confirmed that the lab environment could capture endpoint process activity, centralize the telemetry in Splunk, identify potentially suspicious behavior, and use the available evidence to determine whether the activity required further response.